

ECE.UT.AC.IR ACM تفهیم کار عمیق

ELEARN.UT.AC.IR introduction to
computer system

The C programming
Language

پیکون برنامه نویسی C

گرفتن وقت آزمائشگاه

الگوریتم

پایان دارد

مطالعه انجام باستود و جزئیات معلوم باشد

شرفاء باشد

$$1011_2 = 45_{10}$$

$$13_{10} = 1101_2$$

$$-13_{10} = 11101_2$$

اعلامه من: نسبت بینام Leading bit را برای ضمیمه علامت اختصاص می دهیم:

برای مثبت کار می کنیم

روش مکمل ۱: در این روش از عدد مثبت با شویبت اول را به ویزو قرار می دهیم و اگر منفی با شویبت اول

را ۱ و مکمل بیت ها که می گذاریم به عنوان مثال:

$$13_2 = 1101 \quad +3 \ 0011$$

$$-13_2 = 11101 \quad + -3 \ 1100$$

$$0 \ 1111 \Rightarrow \text{مکمل جمع دل شری}$$

$$0:0000$$

$$0:1111$$

مشکلی که حل شری: چنان روشی برای صفر داریم:

The leading bit represent the sign

0: +

1: -

$$-13_2 = 1 \ 0010$$

$$+ 1 \ 1001$$

$$1 \ 0011$$

when negative invert all bits and
add 1 to the result

$$-0_2 = 1111$$

$$+ 1$$

$$0000$$

مشکلی که صفر بودن حل شری

$$+5 \ 0101$$

$$-5 \ 1101$$

$$0000$$

ignore carry out ← بیت carry می دهیم

$$\begin{array}{r} 4: 0 \ 0100 \\ -2: 1 \ 1110 \\ \hline \text{carryout} \rightarrow 0 \ 0010 = 2 \end{array}$$

مثال: می خواهیم 4-2 را حساب کنیم:

نام بیت از سمت راست: least significant bit → 10010101 ← most significant bit
MSB
LSB

* در روش مکمل، اگر عدد منفی باشد در آیت غایش داده باشیم، برای غایش آن ها در 1 بیت باید 4 تا یک بگذاریم تا مکمل آن ها همان درست و MSB همان یک باشد.

نتیجه:
: Signed Magnitude
: One's complement
: Two's complement

$$\begin{array}{r} -8 \ 11000 \\ + -9 \ 10111 \\ \hline 15 \ 01111 \end{array}$$

Overflow: جمع دو عدد مثبت منفی و یا جمع دو عدد منفی مثبت شود: به عنوان مثال

اعداد اعشاری: $0.1101 \rightarrow 1 \times 0.5 + 1 \times 0.25 + 1 \times 0.125 = 0.875$: همان روش دویسی

* قسمت صحیح و اعشاری را جداگانه تبدیل می کنیم:
 $0.875 = 101.01$
0.5 0.25

IEEE 754 Standard: 32-bit Floatingpoint: اعداد اعشاری را به صورت 32 بیت غایش می دهیم.

$$N = (-1)^{\text{sign}} \times 1.\text{fraction} \times 2^{(\text{exponent} - 127)}$$

1 bit sign 8 bits exponent 23 bits fraction

$$1 \ 01111110 \ 1000 \dots 0 = -1 \times 1.5 \times 2^{(126 - 127)} = -0.75$$

منفی 126 010



موقع نوشتن الگوریتم: باید پیش فرض هارا اصلاح کنیم و مقس کنیم مثلا ۲ قدم مرغی بریل ۳ نفر

الگوریتم
 قدم هاست شخص باشد
 قدم هاست وسط کما پیوتر قابل برابر باشد
 پایان پذیر باشد

فلوچارت: الگوریتم هارا در شکل و فرم خاصی می نویسیم تا بفهمیم در اتفاق در حال رخ دادن است. مثلا شرا هارا
 بالونی می نویسیم با visio

۳ bit = نیل
 بیشتر از ۸ bit = word
 ۸ bit = بیت

مثال عدد باینری ۱۱۱۱۱۰۱۰۱۱۰۰۰۰۱۱ را به مبنای ۱۶ تبدیل کنید.
 Sign ۷ A C 3

مثال: چهار کاراکتر بر اساس کد اسی در حافظه ذخیره شده اند. محتوای این حافظه در مبنای ۱۶ به صورت زیر می باشد.
 ۴ ۹ ۶ ۳ ۵ ۳ ۷ ۵
 I c S p

مثال: عدد ۱۶۵ را از دسیمال به مبنای ۱۱ تبدیل کنید. هجی شود

عدد ۱۱۰۶۵ - در مبنای ۱۰ را به صورت هجی بنویسید و در مبنای ۱۱ تبدیل کنید.

$$۱۱۰۶۵: ۱۰۱۱.۱۱ = ۱.۰۱۱۱۱ \times ۲^۳ = ۱۳۰ - ۱۲۷ = ۳$$

fraction exponent

$$\begin{array}{c} 1 \\ \hline 10000010 \quad 01111000 \end{array}$$

sign 1 bit 23 bit

نکته: با آلیت خود عدد 16 را نمی توان نمایش داد و فقط از صفر تا 15 را می توان نشان داد

* برای extend کردن یک عدد به 32 بیت به 32 آلیت، بیت sign رو extend می کنیم

* شیفت بجای: محاسبه ضرب در 2ⁿ

$$-14 = 11110$$

sign magnitude: 32 بیت برای علامت و بقیه بیت ها برای اندازه است

$$-14 = 10001$$

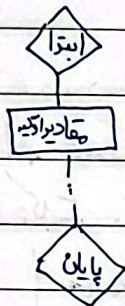
one's complement: عدد منفی را به 1 ها تبدیل می کنیم (بدون تغییر علامت)

$$-14 = 10010$$

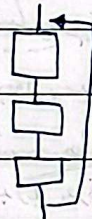
two's complement: یک واحد به one's complement اضافه می کنیم

* overflow: جمع دو عدد مثبت منفی و دو عدد منفی مثبت، بشود

* برای تقریب از روش مکمل دو وجه استفاده می کنیم: $a - b$ ← طراکمل دو کرده و جمع می کنیم



وقتی الگوریتم می نویسیم باید ابتدا مقادیر اولیه ها را تعریف می کنیم

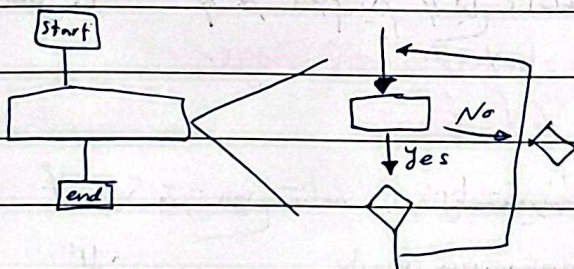


حلقه ها را با ساختن در فلوجارت

start with problem statement

stepwise refinement

decompose task into a few subtasks



* متغیرها باید با under-line و یا با کاراکتر شروع شوند و نمی‌توانند عدد باشند case sensitive

* 31 کاراکتر اول متغیرها را کامپایلر در نظر می‌گیرد

* اگر چند نوع متغیر همزمان عملیات انجام بدهد: smaller is promoted to larger

$x + 4.3 \rightarrow$ if $x = \text{int} \rightarrow$ will convert to double and answer is double

$$x/3 = 1$$

$$x/9.0 = 1.66$$

$$\left. \begin{array}{l} x = 5 \\ x = \text{int} \end{array} \right\}$$

دایم:

Bitwise operators {

$\sim$: bitwise not

$\wedge$: bitwise xor

$\&$: bitwise and

$|$: bitwise or

اول $x++ \rightarrow$ اول مقدار را می‌خواند بعد $+$ $++x \rightarrow$ اول $+$ می‌کند بعد مقدارش را می‌خواند

1- $[], ()$

2- $++$ و $--$ پسوند

3- $*$ و $/$ و $\%$ $\sim$ و $+$

4- $\gg$ و $\ll$

5- $=$ و $==$ و $!=$ و $<$ و $>$ و $<=$ و $>=$ مقایسه

6- $\&$ و $\&\&$

7 $\&\&$ و $||$

ترتیب اولویت اپراتورها:

Decleration: برای نام‌های متغیرها بنویس تا بعد خط اول کد:

`int factorial(int)`

$\Rightarrow$ ما این می‌توانیم که $\Rightarrow$ نام‌های متغیرها بنویسیم که این هست این تابع

اول `call` کنیم تا بعد رو بعد `define` و کدش رو بنویسیم

متغیر Local: متغیرهایی که تعریف می کنیم اگر داخل $\{ \}$ باشند متعلق به اون برکات ها هستند و بعد از بسته شدن آنکوادر از بین می رن و اگر بیرون آنکوادر متغیر با اون اسم باشد مقدارش بیشتر اون مقدار بیرون برکات دوباره. ولی اگر آنکوادر تو اون آنکوادرش باشد مقدارش اون توهم میره.

متغیر Global: متغیر که بیرون هم آنکوادرها تعریف شده باشد و بیرون main باشد و همه جا بهش دسترسی داریم می تونیم عوضش کنیم.

* استفاده از متغیرهای Global ممنوع است *

میکنند $\leftarrow$ define تابع زیری فایل باشد استفاده در یک فایل دیگر بخاطر همین وجود دارد

Declaration of function: وجود تابع رو اظهار می کنیم: `int function (int n);`

Defenition of function: در تابع را می نویسیم: `int function (int n) {
return n * n;
}`

* استفاده از declaration ضروری است ✓ $\leftarrow$ حلالی بنویسند

پس کدها مون به این صورت باید باشند:

`#include <stdio.h>`

`int function (int);` $\rightarrow$ declaration

`int main (void) {`

`printf("%d", function(10));`

`int function (int n) {`

`return n * n;`

`}`

} defenition

* در کدها در زمان ورود گرفتن ابتدای `printf` می نویسیم که چه چیزی را در ورود می گیریم ✓

Declaration: `return_type function_name (int, float)`

type of all arguments

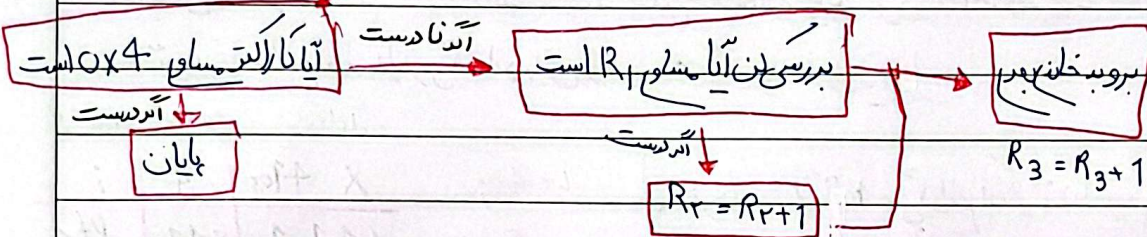
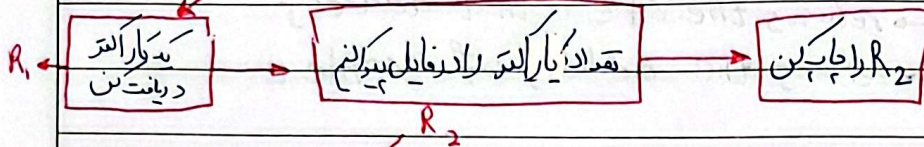
add R_2 to R_6 and store it in R_6 :

: LC-3 ADD instruction

ADD	Dst	Src1	000	Src2
0001	110	010	000	110

شروع → مقدار آدرس و مقدار تکرار → پایان

مثال:



* برای باقی مانده در قلوچارت و سود و کرم نویسی Mod

در زمان تعریف: $\text{char } C[7] = \text{"Tehran"};$ بزرگ $n+1$ خانجیم خانج آخر
مقدار null خواهد داشت. به این کار آدرس $n+1$ جاننا زدیم که آخر
 null نیست و اگر موقع printf تا زمان نباشد null بیسند چاپ می کنند

$\text{char } C[100];$
موقع scanf برای string لازم نیست: $\text{scanf}(\text{"\%.5s"}, C);$

کتابخانه

$\text{char name}[4] = \text{"Ali"};$ → جتا باید ۴ باشد

$\text{int matric}[3][4] = \{ \{ 1, 2, 3, 4 \}, \{ 5, 6, 7, 8 \}, \{ 9, 10, 11, 12 \} \};$
 $\text{int mat}[][3] = \{ 1, 2, 3, 4, 5, 6, 7, 8 \};$

کمترین مقدار ممکن → کامپایلر خودش با 3 پر می کند

تفسیر نخواستیم و مولیٰ را پیدا قابل تفسیر نیست

← جنس مغير

کاتھولک رسالہ

آن آدرس

*p → returns the value in that address

$\&z \rightarrow$ returns the address of variable z

اُدرس خانہ اوّل راہ میں گردانہ دوسرے وقت * اُدرس راہ میں

address	value	name
---------	-------	------

*ptr = *ptr + 1;
i = i + 1; ← محلول

$i = i + 1;$

$\text{int } a[2][2] = \{ \{1, 2\}, \{3, 4\} \}$ و $\{1, 2, 3, 4\}$ ترتيباً بتدريج:

هر دو در این ها یکی است

2. یونیفرم خانوار (Uniform Distribution) $\rightarrow$ $x \in [1, 10]$ int

$$*x = x[0]$$
 ~~$*(x+i) = x[i] \quad (x+i) = x[i]$~~

اگر آرایه دو بجز باشد: int*** می شود یونین بر خاسته آرایه ها

$$* \rho = \dots y_i^c$$

* م ← سمت دیگر عبارت : $\frac{1}{2}$ همان مقدار دادن و مقعیر است که بر آن استار می کند

و نسبت راست عبارت از مقدار را برین گرداند

★ برای نمایش یک string با کاراکتر به $n+1$ مکان نیاز داریم: $\text{char str}[n+1] = \text{"Tehran"}$ ؛ char جای هر کاراکتر $\rho_{+1} = \dots$ کاراکتر آخر Null است

اعداد منفی : sign magnitude
 10101₈ 1011_D -13: 11101
 علامت مقدار علامت مقدار

ones complement: آرمیتی بودگی: -13: 10010

مقدار علامت

جمع زوج عدد قریب = 0

هر چنان توانیش بدی = 0

two's complement: آرمیتی بود عدد مکمل کن و بعد علاوه کن کرد و سپس می‌دانیم.

13: 01101 -13: 10011
 مقدار علامت مقدار علامت

آرخیوایم به علامت دهان غایت عدد را زیاد کنیم bit علامت آن را extend می‌کنیم

باینری: 0.011 = $2^{-2} + 2^{-3} = 0.125 + 0.0625 = 0.1875$
 1.25 = 1.111110110
 علامت مقدار مکمل

1bit sign

8bit exponent

23bit fraction

Floating point representation: اعداد اعشاری را 32 بیت می‌نویسیم

عدد = $(-1)^{\text{sign}} \times 1.\text{fraction} \times 2^{(\text{exponent} - 127)}$
 $-127 \leq i \leq 128$
 عدد را انقدر شیف می‌کنیم که 1 شود

101111110 100000000000000000000000 = 0.75
 sign exponent fraction

بیت 16: 0010 1100
 2 8
 آرمیتی بود بستم چه عدد منفی می‌کنیم: 0x2C یا 2C

* در انتهای رشته‌ها حرفی Null قرار دادم یعنی تا پایان رشته مشخص شود.

* کامپایلر برای نام متغیرها در فقط آنا را که اول را در نظر می‌گیرد

* اول نام متغیرها باید underscore یا کاراکتر باشد ولی در وسط می‌تواند عدد باشد

* Linker مطلب را از کامپایلر و توابع کتابخانه می‌گیرد و فایل را قابل اجرا می‌کند

اجبارها

کامنت

define کردن آن‌ها

عدم استفاده از مقادیر ثابت و

استفاده از default و switch-case ها

نامگذاری متغیرها با نام‌های قابل تشخیص



★ در آنر switch-case ها break فراموش نشود

★ ★ تابع sizeof اندازه متغیر (تعداد بیت های متغیر) را برمی گرداند؛
sizeof(int)=4;
sizeof(char)=1;

★ در آرایه ها دو بصر حتما باید اندازه آن فرد دوم را بنویسیم و بعدا اول را کمترین مقدار دیتواند آن مقدار را ذخیره کند:

int a[][2] = {1, 2, 3, 4, 5}

با 3 برمی نشود

★ Casting: و 3-2 (int) int a =
تغییر نوع

اولویت اپراتورها:

درست راست هم بود همینطور اول مقدار را برمی گرداند بعد متغیر را

تغییری ندارد

0- () و [] 1- ++ و -- (درست چپ 2- ! 3- * و / و %

4- + و - 5- >> و << شیفیت های بیتی 6- مقایسه ها: ==, !=, <, >, <=, >=

7- عملیات های بیتی 8 و 9- عملیات های منطقی 8 & و 11

9- عملیات شرطی: 10- عملیات های انتساب: +=, -=, *=, /=

★ define و return 0 تیم خود را بریز در کدها!

#define MAX_LEN 101

★ موقع ورود گرفتن باید با printf در هر کسی گیرم و توضیح بدهم

نکته رفیق: for (i++, ch=i*2; ...; ...)

?

این دستور ها از دستورها داخل قوراج برای شود و بعد از آن ها

int i=0; char ch;

for (int i=1, ch='A'; i<=4; i++, ch=i*2) printf("%c", ch);

AEKS - خنوبی

* در بیشتر موارد از `Endif` و `Endwhile` در پایان `if` و `while` استفاده کنیم!

* ساختار `Switch-case` برای حالت‌های شرطی: `Switch (expression) {`

`Case Value:`

`/* code goes here */`
`break;`

`default:` → حتما باید برای همه موارد

`/* code goes here */` داشته باشیم

?

برای نمایش تعداد رقم دلخواه: `printf("%.2f", x);`

* اگر تعداد ارقام اعشاری داخل `float` مشخص نشده باشد، با بیش از ۶ رقم اعشاری نمایش داده می‌شود.

* عمل `%` فقط برای اعداد `int` و `long` است و نمی‌تواند در `float` و `double` از آن‌ها استفاده کرد.

باز هم

* اگر بخواهیم تبدیل اعداد ۳ تا ۱۰ گانه را به اعداد ۲ گانه (کارتک) بگیریم و هر یک از کارتک‌ها را در یک متغیر ذخیره کنیم:

ما می‌زنیم `char b;` و این‌ها را می‌زنیم بعد از کارتک‌ها می‌خواهیم برآورد کنیم: `char a[32];`

`scanf("%s", a);` → `scanf` می‌تواند از کارتک‌ها را در `int` در این حالت و ورودی بلقی می‌ماند

`scanf("%c", b);` → `scanf` می‌تواند از کارتک‌ها را در `int` وجود دارد و ورودی نمی‌تواند دیگر به کارتک‌ها (این‌ها می‌شوند ورودی‌ها)

`scanf("%c", b);` روش درست



دولیمینتر است و `scanf`

می‌تواند این را (تبدیل کند و هر دو را در یک متغیر قرار دهد)

★ `printf` و `scanf` ورودی و خروجی ها را به یک بافر می گذارد و ریختن نوع بافر و برداشتن از آن را کارت ترافیک و سیم می توان نامید

کیبورد ← سیستم عامل ← بافر ورودی گرفتن: `stdin`
خروجی → سیستم عامل → بافر خروجی دادن: `stdout`

`*int a;`

`scanf("%d", &a);` ⇒ اگر نام را به درستی وارد کنیم و نوع غی نمی شود

★ نکته: `scanf` از جایی که نتواند دریافت کند به بعد از آن می رود و دریافت نمی کند و به هیچ ورودی ها در `stdin` باقی مانده است

← برای خالی کردن: `while(scanf("%c", &c))`

مهم: `stdin` و `stdout` به صورت `text stream` است و ما هر چه وارد کنیم به

صورت اسکی ذخیره می کند. این `scanf` است که اسکی داخل `stdin` را

به عدد دوباره تبدیل می کند. به خاطر همین برای خالی کردن `stdin` از `%c` استفاده می کنیم

۱۱ خواندن فایل

`int a[max-len];`

`int i=0, j=0`

به `Stream` مربوط به فایل → پوینتر فایل → `FILE *file1, *file2;`

اشاره می کند → از نوع خواندن `file1 = fopen("myData.txt", "r");`

`while (fscanf(file1, "%d", &a[i]) != 1) {`

`i++;`

`}`

`for(j=0; j<i; j++) printf("%d\n", a[j]);`

مثال `scanf` → اعداد داخل فایل را به ترتیب می خواند
که از `stdin` می خواند

برای نوشتن و خواندن از فایل های باینری از `fread` و `fwrite` استفاده کنیم.

`fwrite`:

```
FILE *myfile = fopen("out.txt", "wb");
```

```
char *str = "Hello!";
```

```
fwrite(str, sizeof(char), 5, myfile);
```

```
fclose(myfile);
```

جایی که نوشته می شود
طول تخمین لایه
شماره
تعداد 5 برابر با size

نکته: بعد از `fread` و `fwrite` به این نکته دقت کنید که اولین محل قرار گرفتن فایل در محل جدید قرار می گیرد و کلاً اولین محل قرار گرفتن خوانده می شود. برای جلوگیری از خطا از تابع `fseek` استفاده می کنیم.

`fread`:

```
fread(ptr, sizeof(type), count, FILE *stream);
```

محل ذخیره سازی
اندازه هر یک از المانها
تعداد
پهنای هر فایل

`feof`: gets the pointer to the file, and if we reached character EOF return non-zero value

```
int len = 0;
while (1) {
```

```
    char input = getc();
```

```
    if (feof(input)) break;
```

```
    len++;
```

```
}
```

وقتی به می شود (getc) تا آخر
EOF را بگیرد

`FSEEK`

برای تغییر محل به این گرافیک است.

```
fseek(pointer-to-file, offset, position);
```

FILE *input

موقعیت تغییر
مقدار تغییر مکان
به این گرافیک

→ SEEK-END. آخر فایل

→ SEEK-SET. اول فایل

→ SEEK-CUR. مکان فعلی

به این گرافیک



نکته: $\text{feof}(\text{pointer})$ تنها زمانی false می شود که ما بعد از آخر فایل تلاش به خواندن اطلاعات کنیم: $\text{becomes true only after you try to read past the end of the file}$

وقتی فایل را از نوع w باز می کنیم، داده را over-write می کند، یعنی کاراکترهای جدید را در مکان پوینتر قبلی می کند و پس بقیه فایل را تغییر نمی دهد

حک کردن برای نرسیدن به آخر فایل: $\text{while}(\text{input} = \text{getc}(\text{file})) \neq \text{EOF}$

تیمبر توچ `stdin` می نویسد و توابع `stdin` اطلاعات مورد نیاز را برمی دارند. `stdin` معمولاً 5 تا 6 کاراکتر
 جادارد. `printf` اطلاعات را در `stdout` می نویسد و کاراکتر `\n` را استر باستر `stdout` خالی می‌کند
 و غایش داده می‌شود.

`putchar(c)` → adds one character to `stdout`

`getchar(c)` → Reads one character from `stdin`

در توابع توضیح ورودی باید `\n` را باید ازای نام طین شیم بافر ورودی خالی می‌شود

تازه‌ای که `delimiter` وارد شود → اگر این کار را اجرا کنیم و 1 را وارد کنیم و اینتر
`char c, cc;`
`c = getchar();`
`cc = getchar();` → اینتر کنیم، `cc` برابر اینتر می‌شود

ورودی‌ها در `stdin` وارد می‌شوند و وقتی دلیمتر را می‌زنیم، خود کاراکتر مثل اسپیس یا اینتر نیز در `stdin` نوشته می‌شود و دفعه بعد که
 می‌ایم از `stdin` می‌خوانیم آن را می‌خوانیم.

نکته: برای استفاده از `Single quote` از `'` و برای استفاده از `double quote` از `"` استفاده می‌کنیم

نکته: `printf("Hello %d");` در این خط `printf` از حافظه مقدار عددی را برمی‌دارد
 و در `stdout` قرار می‌دهد.

نکته: تابع `scanf` مقدار ورودی‌هایی را که توانسته است دریافت کند برمی‌گرداند

FILE I/O

- 1) `FILE *input;` → مقدار برآورد شده به بیشتر از نوع `FILE`
- 2) `input = fopen("test.txt", "r");` → `input` یک پوینتر به بافر ایجاد شده برای فایل
- 3) `int a; fscanf(input, "d", &a);` → پوینتر فایل را نوع داده را پس‌دهنده
`r: read w: write a: append`
- 4) `printf("%d", a);` → عدد خوانده شده را فایل چاپ می‌شود
- 5) `fclose(input);` → فایل را حالت `lock` خارج می‌شود



`fopen("", "type")` → `type = r`: open file for reading, return Null if no exist
`type = w`: overwrite if exists, create if doesn't exist
`type = a`: append in end of the file, create if doesn't exist
`type = r+ w+`: for both reading and writing

« آرایه پویا »

طول آرایه در زمان کامپایل معلوم نیست

ورود تابع طول آرایه به پاید است: **malloc**
 باید بررسی کرد که مقدار برگشتی **malloc** نباشد }
 اگر مقدار برگشتی **malloc** **NULL** باشد یعنی برگرد
 اگر مقدار برگشتی **malloc** **NULL** نباشد یعنی برگرد

$\text{type}^* \text{pointer} = (\text{type}^*) \text{malloc}(\text{number} * \text{sizeof}(\text{type}))$

تعداد بایت هارم دهد تعداد بایت خواهم
 مقدار برگشتی **malloc** از نوع **Void** است و باید
 به نوع مورد نظر **cast** شود

free(pointer); Do not forget to free the allocated memory

آدرس پوینتر را عوض نمی کنند فقط آن حافظه را آزاد می کنند

برای اتمام کار در هر جایی باشد (حتی بیرون **main**) متوقف می کنند و با فریبی
 را فلاتش می کنند

exit(EXIT_FAILURE); به سیستم عامل می گویند که خطا است

exit(EXIT_SUCCESS); به سیستم عامل می گویند که بنده بدون خطا بودم

تکته: **malloc** مقدار اولی می عناصر آرایه را صفر می کند و در حال کند

calloc مقدار اولی هم عناصر را صفر می کند

calloc: مثل **malloc** فقط عناصر را صفر می کند

$\text{type}^* \text{pointer} = (\text{type}^*) \text{calloc}(\text{number}, \text{sizeof}(\text{type}))$

تعداد اعضا
 اگر در پوینتر را عوض می کند اگر بتواند لا ابر پوینتر قبلی را اضافه کند همان پوینتر قبلی را **extend** می کند

realloc: برای تغییر طول آرایه استفاده می شود و پوینتر آرایه جدید را بر
 می گرداند و در پوینتر آرایه قبلی را با $\min(\text{new-size}, \text{old-size})$ کم می کند و آرایه جدید

$\text{type}^* \text{pointer} = (\text{type}^*) \text{realloc}(\text{Pointer}, \text{number} * \text{sizeof}(\text{type}))$

cast پوینتر آرایه قدیم
 خودش پوینتر قبلی را **free** می کند =>

`realloc(NULL, size) == malloc(size);` نکتہ:

type casting

`int* p = (int*) malloc (10 * sizeof(int));`

`char* s = (char*) p;` $\Rightarrow$ اب یہاں خانے P اشارہ کی گئی ہے اور اسے `char` کی پینڈروپیش میں پرور: `[39] 5` دارم!

`free(p);`

`free(s);` $\rightarrow$ Error: you can't free a memory twice: double free is an error

ہر دو پینڈروپیش خانے لیسان

data structure

```
struct struct-name {
    int std-No;
    char name[30];
    char family-name[30];
    float grade;
}
```

type جدید

تعریف در سیر دنباله mai است
نامها با دسترسى داشته باشیم

declaration

مى توان مقدار داد
فقط داریم نام را تعریف می کنیم

ساخت یک نسخه از آن $\rightarrow$ struct struct-name std1;

std1.std-No = 810104543;

struct struct-name std2;

std.name = "Iliya"; $\rightarrow$ یا به این حال struct دسترسى پیدا کنیم

struct struct-name a[10]; $\rightarrow$ به آرایه از تایپ جدید

قادر اولی را در آن: struct struct-name std3 = { .grade = 19.0 }

struct struct-name = { 10104543, 12, 3, "Iliya" }

تعریف و استفاده هر زمان: struct weather {

int maxTemp;

int lowTemp;

int currentTemp;

تعریف

} weather-date[10];

استاده

typedef

typedef <data-type> newname

Ex.

typedef int color;

typedef struct weather w-info;

typedef struct std { ... } standard;

pointer to struct:

```
struct name{
```

```
    char name[30];
```

```
    char last-name[30];
```

```
};
```

```
struct name * ptr = &name; → (*ptr) = name
```

معادل نداشتن

نکته: Pass by Pointer و Pass by Value to struct
خودشان یک نوع هستند و مثل آرایه‌ها نیستند



```
weatherData *dayptr;
```

```
dayptr = &days[34];
```

```
*dayptr • high temp = 37; X
```

اولویت بالاتر از * است

```
↳ (*dayptr).high temp = 37 ✓
```

```
↳ dayptr → high temp = 37;
```

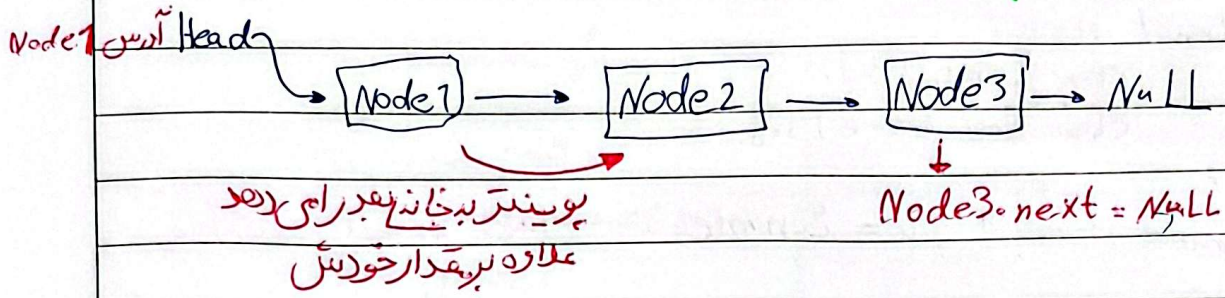
Assignment, Copying

1) struct1 = struct2; → حتی آرایه‌ها نظیر به نظیر کپی می‌کنند

2) memcpy (&day2, &days1, sizeof(days1));

to
from
size

LINK LIST



مثال :

```

struct Node {
    type value;
    struct Node * next;
}
  
```

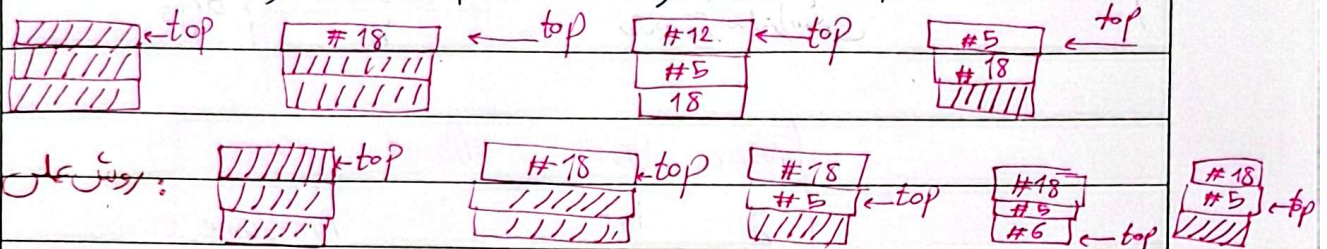
برای ساختن Node جدید: $\text{newNode} = (\text{Node} *) \text{malloc}(\text{sizeof}(\text{Node}))$

تابع‌های delete, add, scanList: ←

« محاسبات کامپیوتر »

Stack: متغیرها در استک ذخیره می شوند. ساختار Last in first out است: LIFO

push: اضافه کردن به سر استک. pop: برداشتن از سر استک



top را جای کنیم و این آدرس سر استک را نگه داریم

Register

دستورها

instruction

به آدرس: R4

global data

متغیرها (تک‌بایه اشاره دارد)

Heap

64 بیت

Register:

پوشش به استک آن‌ها

R6

address to top

R5

runtime stack

به متغیرها (بزرگ) دسترسی سریع‌تر

0xFFFF System Space

فضای Heap: برای allocation مثل malloc, calloc است

« Activation record »

calling function

push new activation record

copy values into arguments

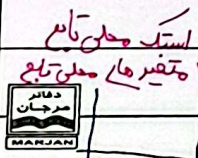
call function

execute code in the function

put result in activation record

pop activation record from stack

متغیر محلی main



before

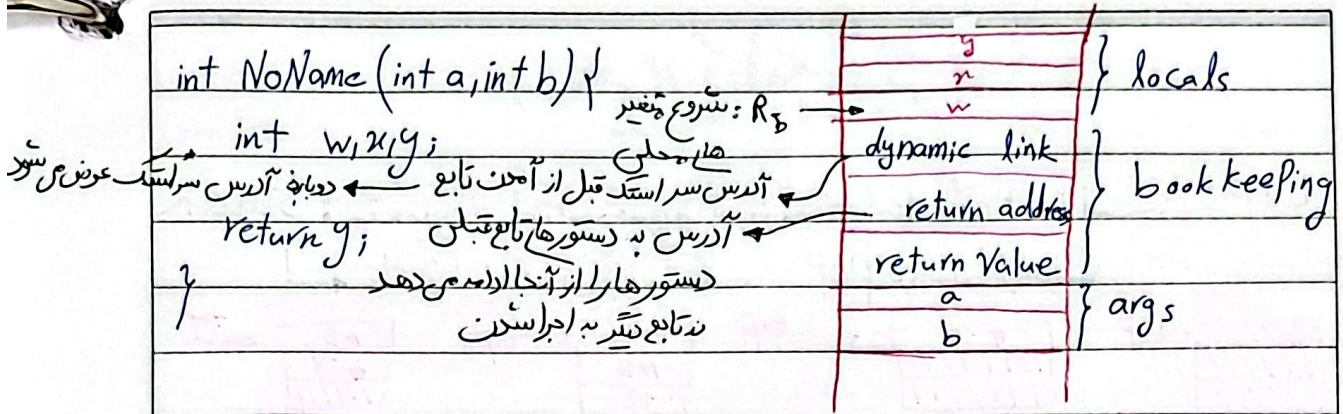


during call



After

رواقع به صورت مقابل است: R5, R6



Name	type	offset	Scope
y	int	1	NoName
x	int	1	NoName
w	int	0	NoName
a	int	-1	NoName
b	int	-2	NoName

بخط هرین بعد از برگرداندن پوینتر متغیر محلی تابع، بعد از تمام شدن تابع آن پوینتر دیگر به مقدار درست اشاره نمی کند

ترانزیستورها: میلیون ها الیو روشن و خاموش داخل سی پی یو

(ساختن مدار از زوج جدول دوستی)

هر جا یک است، بیا اعضا را And کن و بعد هدر را با هم Or بگیر:

A	B	C	D
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

($\bar{A}$ and B and $\bar{C}$)

(A and $\bar{B}$ and C)

($\bar{A}$ and B and $\bar{C}$) or

(A and $\bar{B}$ and C)

جمع:

a_i	b_i	S_i	C_i
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1

به مدار برای این بار و شش بالا

به مدار برای این بار و شش بالا